

NIRMA UNIVERSITY

Institute of Technology

Department of Electronics and Communication Engineering

ESP32 Based Smart Home Monitoring System

*A Mini Project Report submitted in partial fulfilment of
the requirements of the course curriculum*

Submitted by: Nishit Satapara

Roll No.: 23BEC173

Programme: B.Tech, Electronics and Communication Engineering

Institution: Nirma University

Academic Year 2025–2026

Abstract

The growing demand for low-cost home automation has led to widespread interest in microcontroller-based monitoring systems that can be built using commonly available sensors. This project presents the design and implementation of an ESP32 based Smart Home Monitoring System capable of measuring temperature, humidity, gas concentration, rainfall, ambient light level and water tank level in real time. The system uses a DHT11 sensor for temperature and humidity, an MQ-series gas sensor for air-quality and gas-leak detection, a rain detection module, a digital LDR module for light sensing, and an HC-SR04 ultrasonic sensor for water-tank distance measurement. All sensor readings are displayed locally on a 0.96-inch I2C OLED screen and are also published through the ESP32's built-in Wi-Fi web server, allowing the data to be viewed live from any smartphone or laptop browser on the same network through a simple IP-address based dashboard, without depending on any third-party cloud service.

The hardware was assembled on a breadboard, interfaced with the ESP32 development board, and programmed using the Arduino IDE. The complete system was tested by hosting a live HTML dashboard accessible over local Wi-Fi, and sensor values such as temperature, humidity, gas level, rain value, light status and tank distance were successfully observed on a laptop browser as well as on the OLED display simultaneously. The project demonstrates a practical, inexpensive and extensible approach to home environmental monitoring and forms a foundation for future automation features such as relay-controlled appliances, mobile push notifications and cloud logging.

Keywords: ESP32, Smart Home, IoT, DHT11, Ultrasonic Sensor, LDR, Gas Sensor, Web Server, OLED Display.

1 Introduction

The Internet of Things (IoT) has transformed traditional households into intelligent, connected environments capable of monitoring and responding to changes in their surroundings. A smart home system typically integrates sensors, a processing unit, and a communication interface to collect environmental data and present it to the user in an accessible manner. With low-cost system-on-chip modules such as the ESP32 – which combine a dual-core processor with built-in Wi-Fi – it has become feasible for students and hobbyists to build functional IoT systems with minimal hardware investment.

This project focuses on building a Smart Home Monitoring System using an ESP32 microcontroller interfaced with multiple sensors: a DHT11 temperature-humidity sensor, a rain detection module, an MQ-series gas sensor, a digital LDR light sensor, and an HC-SR04 ultrasonic distance sensor. The collected data is displayed locally through an OLED screen and remotely through a self-hosted web dashboard accessible over Wi-Fi. This eliminates dependency on external cloud platforms while still allowing real-time monitoring from a mobile phone or computer.

Most commercial home-automation kits rely on proprietary hubs or paid cloud subscriptions, which increases cost and reduces flexibility for student-level experimentation. The motivation behind this project was to design a self-contained alternative: a single ESP32 board that reads several types of environmental data, shows it instantly on a small onboard display, and also exposes the same data as a webpage on the local network. This approach keeps the system fully functional even without an active internet connection, as long as the monitoring device and the ESP32 share the same Wi-Fi router.

2 Objectives

- To interface multiple environmental sensors (temperature, humidity, gas, rain, light, distance) with the ESP32 microcontroller.
- To display real-time sensor data locally using an I2C OLED display.
- To host a web server on the ESP32 so that sensor data can be viewed on any browser connected to the same Wi-Fi network.
- To design a simple, auto-refreshing HTML dashboard for convenient remote monitoring.
- To test and calibrate sensor logic (e.g. active-low/active-high LDR behaviour) for accurate readings.
- To build a low-cost, expandable platform that can later support automation features such as relay-controlled appliances and alerts.

3 Components Used

Table 1: List of Hardware Components

Component	Function in the System	Qty
ESP32 Development Board	Central processing unit and Wi-Fi controller	1
DHT11 Sensor	Measures ambient temperature and humidity	1
Rain Sensor Module (FC-37/YL-83)	Detects presence and intensity of rainfall	1
MQ Gas Sensor (MQ-2 type)	Detects smoke, LPG and other combustible gases	1
Digital LDR Module	Detects ambient light level (day/night)	1
HC-SR04 Ultrasonic Sensor	Measures distance for water tank level	1
0.96" OLED Display (I2C, SSD1306)	Displays live sensor readings locally	1
Breadboard & Jumper Wires	Prototyping and circuit connections	As required
USB Cable	Programming and power supply to ESP32	1
Laptop/PC	Code development and dashboard viewing	1

4 System Architecture

The system follows a centralized architecture in which the ESP32 acts as the single processing and communication hub. All sensors send analog or digital signals to the ESP32 GPIO pins; the microcontroller processes this data, updates the local OLED display, and simultaneously serves it as an HTML page over Wi-Fi so it can be accessed by any device on the same network.

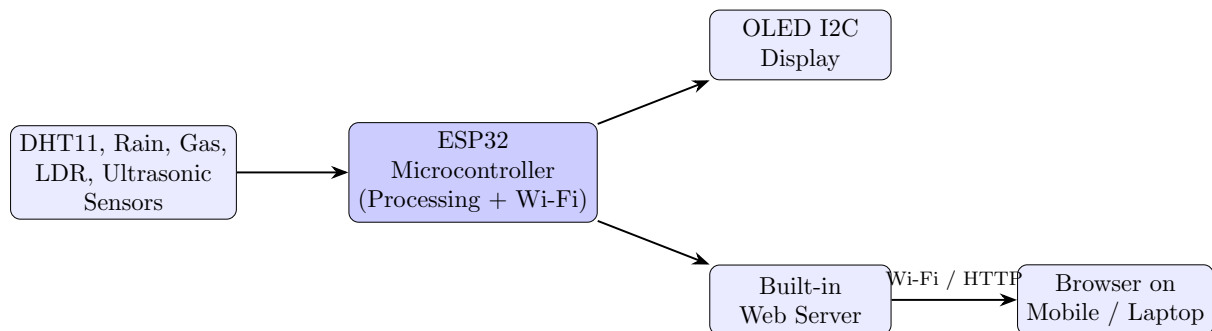


Figure 1: Block diagram of the ESP32 Smart Home Monitoring System

The architecture can be summarized in three layers:

- **Sensing Layer:** Collects raw physical data (temperature, humidity, gas concentration, rainfall, light level, distance).
- **Processing and Communication Layer:** The ESP32 reads sensor values at fixed intervals, formats them, updates the OLED, and serves an auto-refreshing web page.
- **Presentation Layer:** A browser on any Wi-Fi connected device displays the dashboard, refreshing automatically every few seconds.

5 Working Principle

On power-up, the ESP32 initializes all sensor pins, the I2C bus for the OLED display, and connects to the configured Wi-Fi network using the provided SSID and password. Once connected, it starts a lightweight web server on port 80 and prints its assigned IP address to the Serial Monitor.

In the main loop, whenever a client (browser) requests the root page, the ESP32:

1. Reads temperature and humidity from the DHT11 sensor.
2. Reads the analog gas concentration from the MQ sensor and the analog rain intensity from the rain module.
3. Reads the digital output of the LDR module to determine whether the surroundings are bright or dark.
4. Triggers the HC-SR04 ultrasonic sensor and calculates distance using the echo pulse duration, which is used as the water tank level indicator.
5. Updates the OLED display with all six readings.
6. Builds an HTML page embedding these readings inside styled "cards" and sends it back to the requesting browser.

The HTML page contains a meta-refresh tag that reloads the page automatically every two to three seconds, so the dashboard behaves like a live monitoring screen without requiring any additional app. Because the LDR module used in this project produced inverted output (logic HIGH in darkness), the firmware logic was calibrated and corrected during testing to display accurate light status.

6 Hardware and Software Requirements

6.1 Hardware Requirements

Table 2: ESP32 Pin Mapping

Sensor	Sensor Pin	ESP32 GPIO
DHT11	DATA	GPIO 4
HC-SR04 Ultrasonic	TRIG	GPIO 5
HC-SR04 Ultrasonic	ECHO	GPIO 18
Digital LDR Module	DO	GPIO 32
Rain Sensor	AO	GPIO 34
MQ Gas Sensor	AO	GPIO 35
OLED Display	SDA	GPIO 21
OLED Display	SCL	GPIO 22

6.2 Software Requirements

Table 3: Development Tools and Libraries

Tool / Library	Purpose
Arduino IDE	Code editing, compilation and uploading to ESP32
ESP32 Board Package (Espressif)	Board support for ESP32 development
DHT Sensor Library (Adafruit)	Reading temperature and humidity from DHT11
Adafruit Unified Sensor	Dependency required by the DHT library
Adafruit SSD1306	Driving the I2C OLED display
Adafruit GFX Library	Graphics primitives for the OLED display
WiFi.h / WebServer.h (ESP32 core)	Wi-Fi connection and HTTP web server hosting

7 Implementation

The hardware was assembled on a breadboard as shown in Figure 2. The DHT11, rain sensor, MQ gas sensor, digital LDR module and OLED display were wired to the ESP32 according to the pin mapping in Table 3, with sensors operating on either 3.3 V or 5 V supply as per their datasheet requirements, and all ground connections tied to a common ESP32 ground rail. The HC-SR04 ultrasonic sensor was placed to measure the distance to a reference surface, simulating water tank level sensing.

The firmware was developed in the Arduino IDE in incremental stages: first individual sensors were tested and verified through the Serial Monitor, then the OLED display integration was added, and finally a self-hosted HTTP web server was implemented to replace an initially considered Blynk cloud-based dashboard. This local web-server approach was chosen because it removes dependency on third-party cloud accounts and works entirely over the local Wi-Fi network. During testing, it was found that the LDR module's digital output was inverted compared to the expected logic, so the firmware condition was corrected to LOW = Bright and HIGH = Dark.

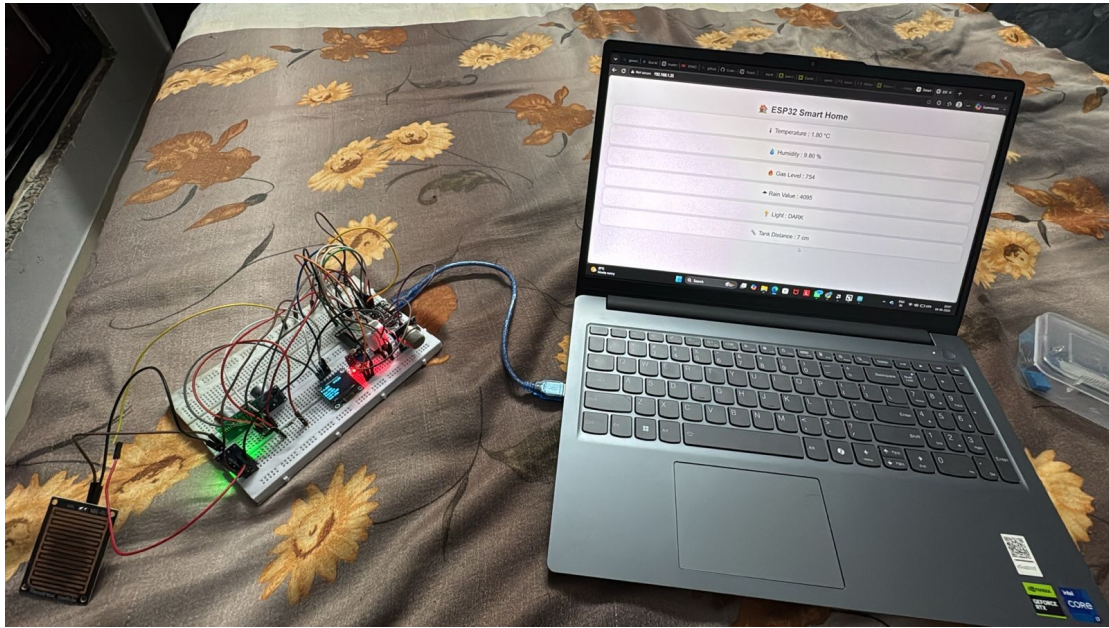


Figure 2: Hardware prototype on breadboard (left) interfaced with ESP32, and the live web dashboard accessed over Wi-Fi on a laptop browser (right), showing real-time temperature, humidity, gas level, rain value, light status and tank distance.

7.1 Development and Testing Stages

The firmware was built and verified in incremental stages rather than as a single block of code, which made debugging straightforward:

Table 4: Stage-wise Development Process

Stage	Task	Verification Method
1	Individual sensor testing (DHT11, rain, gas, LDR, ultrasonic)	Serial Monitor print statements for each sensor
2	OLED display integration	Static text and then live sensor values shown on screen
3	Wi-Fi connectivity	IP address printed on Serial Monitor after successful connection
4	Web server and HTML dashboard	Dashboard loaded and verified on laptop and mobile browsers
5	LDR logic correction	Compared printed digital values against actual lighting condition
6	Full system integration	All sensors, OLED and web dashboard tested together continuously

This staged approach ensured that any wiring or logic error could be isolated to a specific module before integrating it with the rest of the system, reducing overall debugging time.

8 Results and Observations

The complete system was powered on and connected to the local Wi-Fi network. The Serial Monitor confirmed successful Wi-Fi connection and displayed the assigned IP address, which was then entered into a laptop browser to load the live dashboard. The dashboard refreshed automatically and displayed updated sensor readings, confirming correct end-to-end operation of sensing, processing and web hosting.

Table 5: Sample Sensor Readings Observed on the Web Dashboard

Parameter	Observed Value	Remark
Temperature	1.80 °C	DHT11 reading; low value due to indoor test placement
Humidity	9.80 %	Indoor low-humidity test condition
Gas Level	754	Raw ADC value from MQ sensor (no leak detected)
Rain Value	4095	Raw ADC value indicating dry condition
Light Status	DARK	Confirmed correct logic after LDR calibration
Tank Distance	7 cm	Distance from HC-SR04 to reference surface

Observations:

- All six sensor parameters were successfully transmitted from the ESP32 to the browser dashboard without noticeable lag, confirming reliable Wi-Fi based local communication.
- The OLED display and the web dashboard showed matching values at all times, validating consistent data handling in the firmware.
- The digital LDR module initially reported inverted readings; after correcting the logic condition in firmware, the light status matched the actual ambient lighting.
- The dashboard auto-refresh (HTTP meta-refresh) provided a near real-time monitoring experience without requiring a dedicated mobile application.
- Raw ADC values for the gas and rain sensors are usable for relative comparison and threshold-based alerts but would benefit from calibration against known reference conditions for absolute accuracy.

9 Advantages

- **Low cost:** Built using inexpensive, easily available sensor modules and a single ESP32 board.
- **No cloud dependency:** Data is served directly from the ESP32 over the local network, avoiding subscription or third-party cloud services.
- **Dual monitoring:** Sensor data is visible both locally (OLED) and remotely (web dashboard) at the same time.
- **Multi-parameter monitoring:** A single system covers temperature, humidity, gas, rain, light and water level.
- **Platform independent:** The dashboard can be accessed from any device with a web browser, without installing a dedicated app.
- **Scalable design:** Additional sensors or actuators can be added with minimal changes to the existing code structure.

10 Applications

- Residential environmental monitoring (temperature, humidity, air quality).

- Gas leakage detection and early safety alerts in kitchens or storage rooms.
- Automatic lighting control based on ambient light conditions.
- Rainfall detection for automated window or shade closing systems.
- Water tank level monitoring to prevent overflow or dry-running of pumps.
- Low-cost IoT learning platform for students exploring embedded systems and web technologies.

11 Limitations

- Sensor readings (especially from the MQ gas sensor and rain module) are raw ADC values and require calibration against known reference conditions for absolute accuracy.
- The DHT11 sensor has a relatively low sampling rate and limited accuracy compared to more advanced sensors such as the DHT22.
- The web dashboard refresh relies on a simple HTTP meta-refresh rather than a persistent connection, which is adequate for monitoring but not suited for very high-frequency updates.
- The system currently supports monitoring only; it does not yet include automated actuation such as relay-controlled appliances.
- Access to the dashboard is restricted to devices connected to the same local Wi-Fi network as the ESP32.

12 Future Scope

- Integration of a relay module to automatically control lights, fans or pumps based on sensor thresholds.
- Addition of a buzzer for audible gas-leak and rain alerts.
- Use of a servo motor to automatically close windows when rain is detected.
- Cloud logging (e.g., Firebase or ThingSpeak) for historical data storage and trend analysis.
- Mobile push notifications for critical events such as gas leaks or tank overflow.
- Addition of a PIR motion sensor and ESP32-CAM module for security monitoring.
- Sensor calibration routines to convert raw ADC values into standard physical units.

13 Conclusion

This project successfully demonstrates a functional, low-cost Smart Home Monitoring System built around the ESP32 microcontroller. By integrating temperature-humidity, gas, rain, light and ultrasonic distance sensors with an OLED display and a self-hosted web server, the system provides real-time environmental monitoring accessible both locally and remotely over Wi-Fi, without dependence on external cloud platforms. The successful testing of the live dashboard, along with correct sensor calibration, confirms the practical feasibility of the design. The modular nature of the implementation makes it a strong foundation for extending into a fully automated smart home system with actuation, alerts, and cloud connectivity in future iterations.

14 References

1. Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems Co., Ltd.
2. Adafruit Industries, *DHT11/DHT22 Sensor Documentation*, Adafruit Learning System.

3. Adafruit Industries, *SSD1306 OLED Display Library Documentation*, Adafruit Learning System.
4. Texas Instruments / Generic Datasheet, *HC-SR04 Ultrasonic Ranging Module*.
5. Generic Sensor Datasheet, *MQ-2 Gas Sensor Module*.
6. Arduino Official Documentation, <https://www.arduino.cc/reference/en/>.
7. Espressif Systems, *ESP32 Arduino Core Documentation*, <https://docs.espressif.com/>.